SOC STAJ SUNUMU

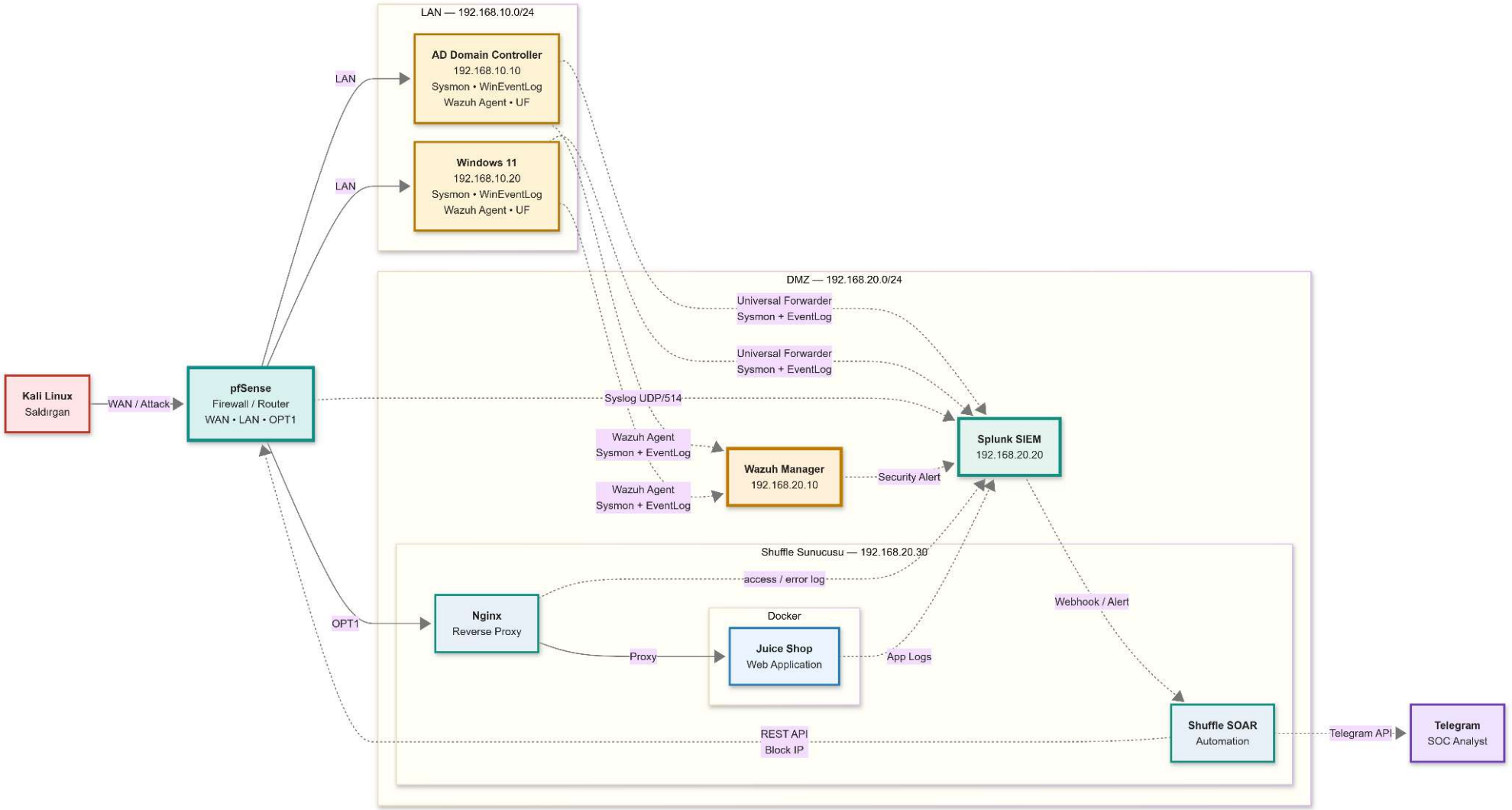
SOC Mini Lab: Tespit ve Otomasyon

Splunk ile Tespit, Shuffle ile SOAR Otomasyonu

3 Case Study · Splunk SPL · Shuffle SOAR · pfSense Otomatik Engelleme · Telegram Bildirimi

Fatih Kaya
SOC Stajyeri · Güvenlik Operasyonları Merkezi · Ağustos 2026

Lab Mimarisi: Ağ Topolojisi ve Log Akışı



Not: Bu ortamda DMZ segmentine klasik anlamda WAN'dan doğrudan port yönlendirmesi yapılmamıştır; test kolaylığı için pfSense üzerinden erişime izin verilmiştir. Güvenlik araçlarının ve hedef sunucunun bulunduğu bu segment, mimarideki rolü nedeniyle yine de DMZ olarak adlandırılmıştır.

CASE STUDY 1

Directory Scanning (Recon)

T1595.003

Kali'den Juice Shop web uygulamasına (192.168.20.30:8080) gobuster ile dizin/endpoint taraması yapıldı. Bu, saldırı zincirinin ilk adımıdır; saldırgan henüz bir zafiyeti istismar etmeden önce saldırı yüzeyini haritalar.

Splunk'ta Nasıl Tespit Ettik

```
index=juiceshop sourcetype=access_combined
| where NOT match(uri, "\.(jpg|jpeg|png|svg|css|js|ico|woff|woff2|gif|map)$")
| where NOT match(uri, "^/socket\.io")
| where NOT match(uri, "^/rest/user/whoami")
| bin _time span=10s
| stats count dc(uri) as unique_uris by clientip, _time
| where count > 10 AND unique_uris > 8
```

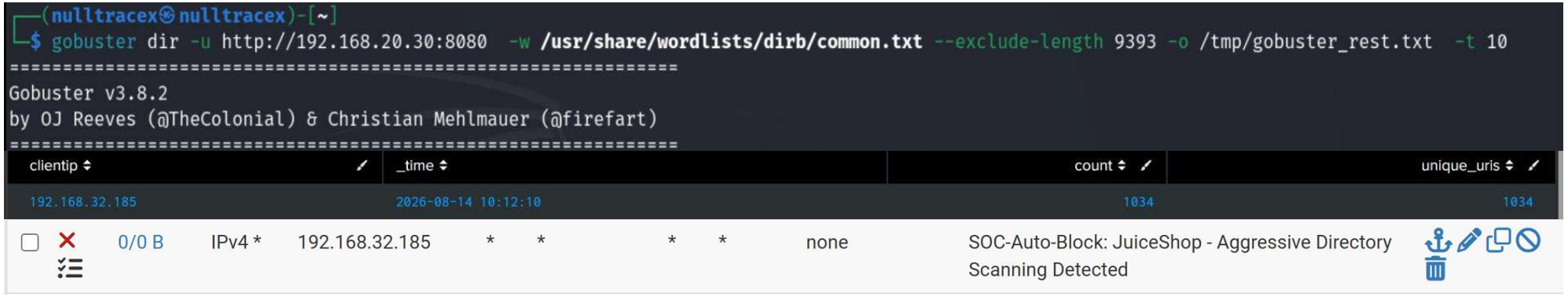
dc(uri) yani farklı URI sayısı esas metrik: normal bir kullanıcı aynı sayfaları ister, tarama araçları kısa sürede çok sayıda FARKLI endpoint dener. Bu davranışsal/volumetrik bir tespit yaklaşımıdır.

- ✦ count değil dc(uri) kritik: tekrar eden istekler değil, çeşitlilik aranıyor
- ✦ Statik dosyalar (jpg, css, js) ve socket.io/whoami gibi sürekli çalışan endpoint'ler filtrelendi
- ✦ 10 saniyelik pencere, otomatik araç hızını normal kullanıcı davranışından ayırt eder

KARŞILAŞILAN ZORLUK: Erişim, Wildcard ve Yanlış Pozitif

Kali başlangıçta DMZ'ye erişemiyordu; pfSense WAN'da eksik olan pass kuralı ve açık olan 'block private networks' ayarı düzeltildi. Juice Shop bir SPA olduğu için her bilinmeyen URL'ye 200 dönüyordu, gobuster'da --exclude-length ile çözüldü. İlk versiyonda statik asset istekleri (resim, css, js) recon alarmını yanlışlıkla tetikliyordu; bu istekler filtrelenerek false positive giderildi.

KANIT / EKRAN GÖRÜNTÜLERİ



CASE STUDY 2

SQL Injection

T1190

Juice Shop login formuna ' OR 1=1 -- payload'ı ile SQL Injection denendi ve admin JWT token ele geçirildi. Bu, saldırganın doğrudan bir yetkilendirme kontrolünü atlatarak sisteme erişim kazandığı ilk erişim (initial access) adımıdır.

Splunk'ta Nasıl Tespit Ettik

```
index=juiceshop sourcetype=access_combined uri="/rest/user/login"
| rex field=_raw "body=\"(?<request_body>[^\"]*)\""
| regex request_body="(?i)('|--|\bOR\b\s+\d+\s*=\s*\d+|\bUNION\b.*\bSELECT\b)"
| table _time, clientip, request_body, status
```

SQLi payload'ı POST gövdesinde gider, URL'de değil. Standart nginx access_combined formatı bunu göstermez; bu yüzden özel bir loglama katmanı gerekti.

- ✦ İki katmanlı çözüm: nginx log_format'a $request_body eklendi, Splunk'ta regex ile arandı
- ✦ Regex SQLi'ye özgü kombinasyonları arıyor (OR 1=1, UNION SELECT), tek apostrof yeterli değil
- ✦ Bu bir imza tabanlı tespit yaklaşımıdır: bilinen kötü amaçlı desenler aranıyor

KARŞILAŞILAN ZORLUK: Body Loglama ve Otomatik Engelleme Kurulumu

Varsayılan nginx log_format POST gövdesini içermez; özel log_format ve client_body_buffer_size ile çözüldü. Body içindeki geçersiz JSON escape karakterleri Shuffle'da parse hatasına yol açtı, sanitize fonksiyonu eklendi. pfSense REST API ile otomatik engelleme kurulurken, pfSense kurallarının ilk eşleşen kazanır mantığıyla çalıştığı ve otomatik oluşturulan kuralın listenin altına düşüp hiç eşleşmediği fark edildi; sabit ve en üstte duran tek bir kuralı PATCH ile güncelleme mimarisine geçildi.

KANIT / EKRAN GÖRÜNTÜLERİ

```
┌──(nulltracex㉿nulltracex)-[~]
└─$ curl -s -X POST http://192.168.20.30:8080/rest/user/login -H "Content-Type: application/json" -d '{"email":"'"'"' OR 1=1 --","password":"x"}'
|python3 -m json.tool 2>/dev/null
{
    "authentication": {
        "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoxLCJ1c2VybmFtZSI6IiIsImVtYWlsIjoiYWRtaW5AanVpY2Utc2gub3AiLCJwYXNzd29yZCI
```

_time	clientip	request_body	status
2026-08-14 14:46:32	192.168.32.185	{\x22email\x22:\x22' OR 1=1 --\x22,\x22password\x22:\x22x\x22}	200
2026-08-14 14:14:10	192.168.32.185	{\x22email\x22:\x22' OR 1=1 --\x22,\x22password\x22:\x22x\x22}	200

☐ ✕	0/0 B	IPv4 *	192.168.32.185	*	*	*	*	none	SOC-Auto-Block: JuiceShop - SQL Injection Login Attempt Detected	⚓ ✎ ⧉ ⊘ 🗑

CASE STUDY 3

Brute Force

T1110

Active Directory'e karşı bir kullanıcı hesabına yönelik tekrarlanan başarısız oturum açma denemeleri (password guessing) yapıldı. Başarılı bir deneme, saldırganın domain içine ilk girişini sağlar.

Splunk'ta Nasıl Tespit Ettik

```
index=wineventlog EventCode=4625
| bin _time span=5m
| stats count by TargetUserName, IpAddress, _time
| where count > 5

index=wineventlog (EventCode=4625 OR EventCode=4624)
| where TargetUserName="Administrator"
| table _time, EventCode, TargetUserName, IpAddress
```

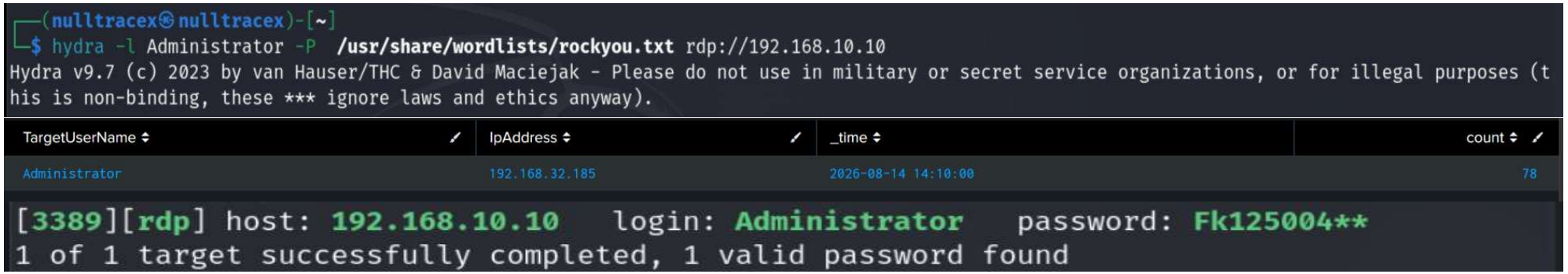
Tek başına eşik aşımı yeterli değildir. İkinci sorgu ile 4625 (başarısız) sonrasında aynı kullanıcı için 4624 (başarılı) geli p gelmediği korelasyonu yapılır; bu, 'sadece deneme' ile gerçek ihlali ayırt eden kritik adımdır.

- ✦ IpAddress alanı sonradan eklendi: saldırganın kaynağını bilmeden containment kararı alınamaz
- ✦ 5 dakikalık pencere ve 5'ten fazla deneme eşiği, meşru unutkanlıkla karışmayacak şekilde ayarlandı
- ✦ Bu, anomali ve imza tabanlı yaklaşımın bir aradalığıdır: hem hacim hem sonuç (başarılı login) kontrol edilir

KARŞILAŞILAN ZORLUK: IP Formatı ve Alert Yapılandırması

Windows event'lerindeki IpAddress alanı IPv4-mapped IPv6 (::ffff:x.x.x.x) formatında geldi, regex öncesi temizlendi. Splunk'ta 'Once ' tetikleme seçeneği ile alan bazlı 'Suppress' özelliğinin birlikte kullanılamadığı görüldü; 'For each result' seçilerek çözüldü. Hydra'nın RDP modülü deneysel o lduğu için bazı FreeRDP bağlantı hataları alındı, ancak doğru kimlik bilgisi yine de başarıyla tespit edilip raporlandı.

KANIT / EKRAN GÖRÜNTÜLERİ



Shuffle Workflow: Tek Zincir, Üç Case Study

- ✦ Tek merkezi Shuffle workflow'u; her 3 case study için aynı Webhook, Execute Python, Http (Telegram + pfSense) yapısını kullanıyor
- ✦ Python kodu, search_name alanına göre hangi alert'in tetiklendiğini ayırt edip MITRE etiketini ve koşullu engellemeyi (should_block) otomatik belirliyor

PYTHON KODU (sadeleştirilmiş, 3 case study'ye özel)

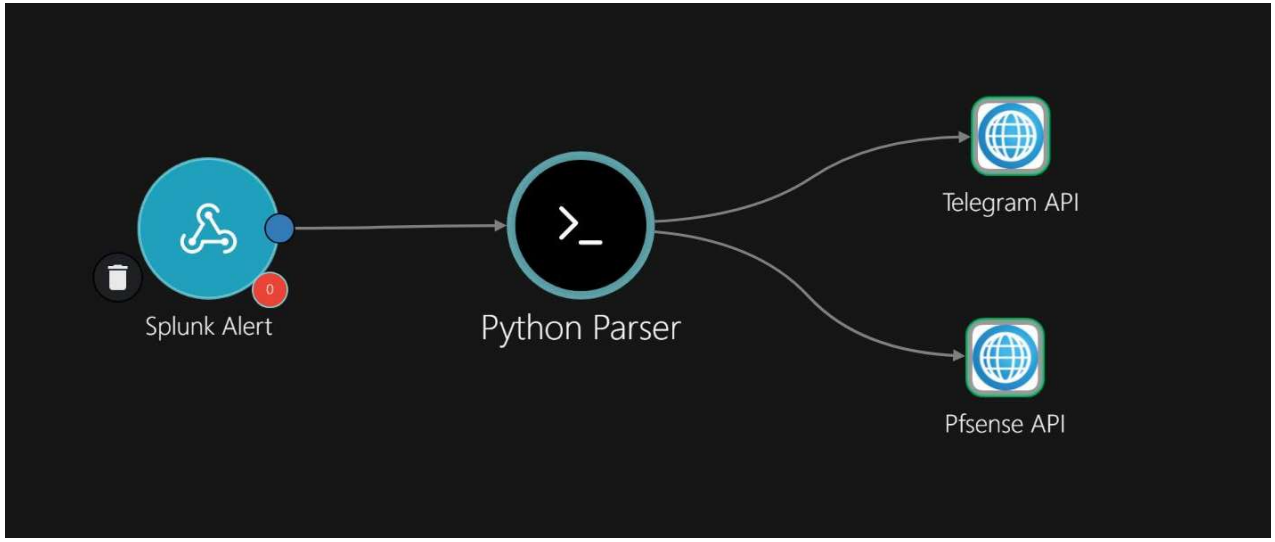
```
import json, re
raw_data = """$exec"""
data = json.loads(raw_data)
alert_name = data.get("search_name", "Bilinmeyen Alert")
result = data.get("result", {})

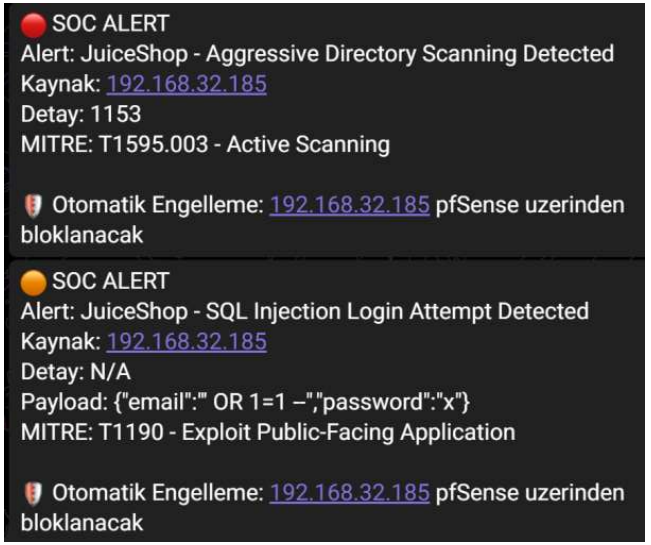
attacker_ip = (result.get("clientip") or result.get("IpAddress") or "").replace("::ffff:", "")
is_valid_ip = bool(re.match(r"^[0-9]{1,3}(\.[0-9]{1,3}){3}$", attacker_ip))
should_block, block_ip = False, ""

if "Scanning" in alert_name:
    mitre = "T1595.003 - Active Scanning"
    if is_valid_ip: should_block, block_ip = True, attacker_ip
elif "SQL Injection" in alert_name:
    mitre = "T1190 - Exploit Public-Facing Application"
    if is_valid_ip: should_block, block_ip = True, attacker_ip
elif "Brute Force" in alert_name:
    mitre = "T1110 - Brute Force"
    if is_valid_ip: should_block, block_ip = True, attacker_ip

# ... Telegram mesaji ve pfSense govdesi bu degiskenlerle olusturulur
```

KANIT / EKRAN GÖRÜNTÜLERİ





Teşekkürler